
Regresión y Clustering E-Commerce

Predicción y segmentación sobre datos de ventas y clientes de un comercio electrónico

Autor

Saúl Basconcillos Corral

24 de mayo de 2026

Índice

1. Introducción	4
1.1. Objetivos del proyecto	4
2. Exploración	5
2.1. Carga y visualización global del dataset	5
2.2. Valores duplicados	6
2.3. Valores nulos	7
2.4. Valores únicos	8
2.5. Variables numéricas	9
2.6. Variables categóricas	9
2.7. Análisis del identificador de factura (InvoiceNo)	10
2.8. Análisis del código de producto (StockCode)	11
2.9. Análisis de la cantidad (Quantity)	13
2.10. Análisis del precio unitario (UnitPrice)	13
2.11. Análisis de la fecha de factura (InvoiceDate)	13
2.12. Análisis del identificador de cliente (CustomerID)	14
2.13. Análisis del país (Country)	14
2.14. Detección de outliers	14
2.14.1. Outliers en Quantity	15
2.14.2. Outliers en UnitPrice	15
3. Limpieza de Datos	17
3.1. Valores duplicados	17
3.2. Identificadores de factura (InvoiceNo)	18
3.3. Identificadores de producto (StockCode)	18
3.4. Descripciones de producto	18
3.4.1. Descripciones inválidas	18
3.4.2. Descripciones duplicadas	18
3.5. Valores negativos y ceros en Quantity	19
3.6. Valores negativos y ceros en UnitPrice	19
3.7. Nulos en CustomerID	19
3.8. Tratamiento de outliers	19
3.9. Resumen de la limpieza	20
3.10. Exportación del dataset limpio	22
3.10.1. Dataset para regresión	22
3.10.2. Dataset para clustering	22

4. Tratamiento de datos para Clustering	23
4.1. Carga y verificación del dataset	23
4.2. Selección de variables	24
4.3. Corrección del tipado	24
4.4. Generación de nuevas variables	25
4.5. Eliminación de valores extremos	26
4.6. Normalización	27
4.7. Exportación del dataset transformado	27
5. Modelo K-Means	29
5.1. Carga de datos	29
5.2. Preparación de los datos	29
5.3. Número óptimo de clusters	29
5.4. Entrenamiento del modelo	31
5.5. Evaluación del modelo	31
5.6. Perfiles de clusters	31
5.7. Visualización de clusters	32
5.8. Segmentación de clientes	33
6. Modelo DBSCAN	35
6.1. Carga de datos	35
6.2. Preparación de los datos	35
6.3. Búsqueda de parámetros óptimos	35
6.4. Entrenamiento del modelo	36
6.5. Evaluación del modelo	37
6.6. Visualización de clusters	37
6.7. Perfiles de clusters	38
6.8. Segmentación de clientes	39
7. Tratamiento de datos para Regresión	41
7.1. Carga y verificación del dataset	41
7.2. Selección de variables	42
7.3. Corrección del tipado	42
7.4. Generación de nuevas variables	42
7.5. Eliminación de outliers	43
7.6. Preprocesado de variables categóricas	45
7.7. Normalización	45
7.8. División en conjuntos de entrenamiento, validación y test	45

8. Modelo Random Forest	46
8.1. Carga del dataset	46
8.2. Entrenamiento del modelo	46
8.3. Evaluación del modelo	47
8.4. Importancia de variables	48
8.5. Visualización de predicciones	49
9. Modelo XGBoost	50
9.1. Carga del dataset	50
9.2. Entrenamiento del modelo	50
9.3. Evaluación del modelo	51
9.4. Importancia de variables	52
9.5. Visualización de predicciones	53
10. Conclusiones	54
10.1. Sobre el documento	56
10.2. Repositorio y código fuente	56

1. Introducción

Este proyecto aplica técnicas de aprendizaje automático supervisado y no supervisado sobre un dataset real de ventas de una plataforma de comercio electrónico (*E-Commerce*). El análisis se divide en dos grandes bloques: regresión y clustering, con el objetivo de extraer valor de los datos tanto a nivel predictivo como descriptivo.

El dataset utilizado (`data.csv`) contiene registros de transacciones de una tienda online registradas entre diciembre de 2010 y diciembre de 2011, incluyendo información sobre facturas, productos, cantidades, precios, clientes y países.

1.1. Objetivos del proyecto

Los objetivos principales de este análisis son los siguientes:

- **Predicción de ventas (Regresión):** Generar una predicción de ventas desde el 9 de noviembre de 2011 hasta el 9 de diciembre de 2011, y comprobar los resultados obtenidos. Los datos de entrenamiento abarcan desde el 1 de diciembre de 2010 hasta el 8 de noviembre de 2011.
- **Segmentación de clientes (Clustering):** Clasificar a los clientes en función de sus características y comportamientos de compra, utilizando técnicas de clustering para identificar segmentos con patrones similares.

Para alcanzar estos objetivos, el proyecto se estructura en las siguientes fases:

1. **Exploración inicial:** Análisis preliminar del dataset, identificando su estructura, tipos de variables y posibles valores faltantes o erróneos.
2. **Limpieza de datos:** Tratamiento de duplicados, nulos, valores inválidos y outliers para obtener un dataset limpio y fiable.
3. **Tratamiento y generación de variables:** Creación de nuevas variables adaptadas a cada modelo, diferenciando el tratamiento para regresión y para clustering.
4. **Entrenamiento y evaluación de modelos:** Entrenamiento de los modelos de regresión y clustering, evaluando su rendimiento con las métricas apropiadas.

2. Exploración

En esta sección se realiza una exploración inicial del dataset de ventas *E-Commerce*, con el fin de comprender su estructura, detectar anomalías y sentar las bases para la posterior limpieza y tratamiento de los datos. Las librerías utilizadas son **Pandas** para la manipulación y análisis de datos, y **Matplotlib** para la visualización.

2.1. Carga y visualización global del dataset

El dataset se ha cargado forzando la codificación ISO-8859-1, ya que la codificación por defecto (UTF-8) generaba errores al encontrar caracteres incompatibles. Esto puede haber provocado que algunos caracteres especiales no se interpreten correctamente.

Número de filas: 541909

Número de columnas: 8

InvoiceNo	str
StockCode	str
Description	str
Quantity	int64
InvoiceDate	str
UnitPrice	float64
CustomerID	float64
Country	str

El dataset cuenta con **541.909 filas** y **8 columnas**. Los formatos objetivo para cada variable son los siguientes:

- **InvoiceNo** (String): Identificador único de cada factura.
- **StockCode** (String): Identificador único de cada producto.
- **Description** (String): Descripción del producto.
- **Quantity** (Numérico): Cantidad de cada producto por factura.
- **InvoiceDate** (Fecha): Fecha de la factura.
- **UnitPrice** (Numérico): Precio unitario de cada producto.
- **CustomerID** (String): Identificador único de cada cliente.
- **Country** (String): País del cliente.

Visualizando las primeras filas se comprueba que `InvoiceDate` está representada como cadena de texto y deberá convertirse a tipo fecha. Las variables `Quantity`, `UnitPrice` y `CustomerID` son cargadas como numéricas por defecto, si bien `CustomerID` es un identificador categórico y deberá tratarse como tal.

Cuadro 1: Primeras 5 filas del dataset (columnas de identificación y cantidades).

<code>InvoiceNo</code>	<code>StockCode</code>	<code>Description</code>	<code>Qty</code>	<code>InvoiceDate</code>	<code>UnitPrice</code>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55
536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39

Cuadro 2: Primeras 5 filas del dataset (columnas de cliente y país).

<code>CustomerID</code>	<code>Country</code>
17850.0	United Kingdom
17850.0	United Kingdom
17850.0	United Kingdom
17850.0	United Kingdom
17850.0	United Kingdom

2.2. Valores duplicados

Las filas duplicadas no aportan valor al análisis y producen sesgos al repetir la misma información aumentando el peso de ciertos datos.

Cantidad de filas duplicadas: 5268

Porcentaje de filas duplicadas: 0.97%

Porcentaje de filas no duplicadas: 99.03%

Se detectan **5.268 filas duplicadas**, lo que representa aproximadamente el **1 %** del

total. Dado su bajo porcentaje, su eliminación no afectará de forma significativa a la muestra.

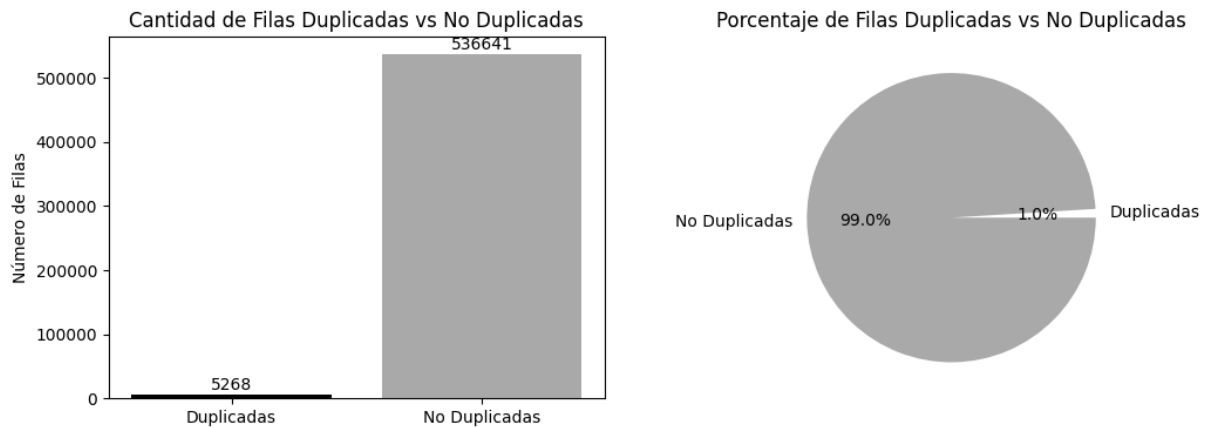


Figura 1: Cantidad y porcentaje de filas duplicadas frente a no duplicadas.

2.3. Valores nulos

El análisis de valores nulos revela que únicamente dos columnas presentan datos ausentes. Habrá que comprobar si pueden recuperarse de alguna forma o deben ser eliminados.

Cuadro 3: Valores nulos por columna del dataset original.

Columna	Valores nulos	Porcentaje
InvoiceNo	0	0,00 %
StockCode	0	0,00 %
Description	1.454	0,27 %
Quantity	0	0,00 %
InvoiceDate	0	0,00 %
UnitPrice	0	0,00 %
CustomerID	135.080	24,93 %
Country	0	0,00 %

- **Description:** 1.454 valores nulos, posiblemente por productos sin descripción asignada.
- **CustomerID:** 135.080 valores nulos (aproximadamente el **25 %** del total), lo que puede deberse a compras realizadas sin registro de cliente. El resto de columnas no presentan ningún valor nulo.

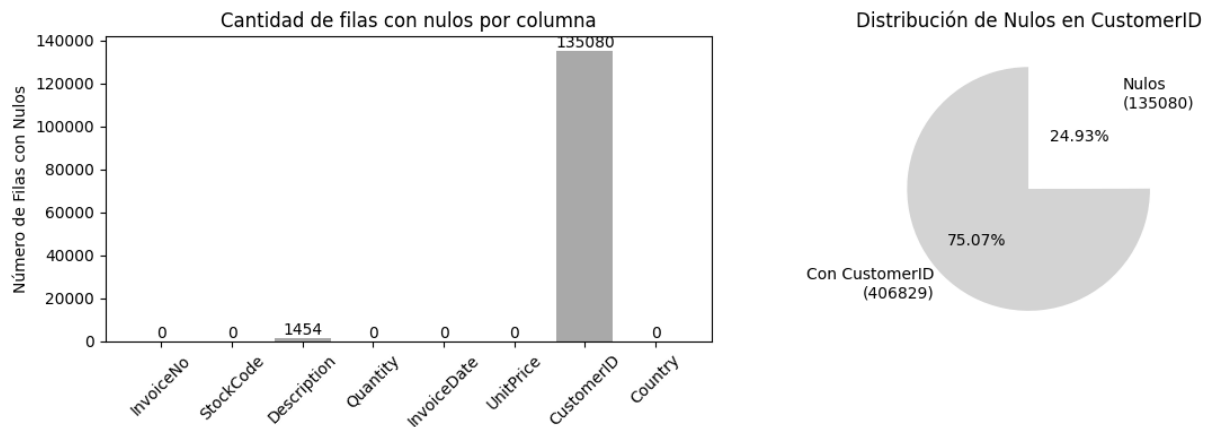


Figura 2: Cantidad de valores nulos por columna y distribución de nulos en CustomerID.

2.4. Valores únicos

El análisis de la cantidad de valores únicos por variable permite extraer las siguientes conclusiones:

Número de valores únicos en InvoiceNo:	25900		Totales:	541909
Número de valores únicos en StockCode:	4070		Totales:	541909
Número de valores únicos en Description:	4223		Totales:	541909
Número de valores únicos en Quantity:	722		Totales:	541909
Número de valores únicos en InvoiceDate:	23260		Totales:	541909
Número de valores únicos en UnitPrice:	1630		Totales:	541909
Número de valores únicos en CustomerID:	4372		Totales:	541909
Número de valores únicos en Country:	38		Totales:	541909

- Múltiples filas comparten el mismo InvoiceNo, lo que indica que dentro de una misma factura pueden aparecer varios productos distintos.
- La cantidad de valores únicos en Description (4.223) supera a la de StockCode (4.070), lo que sugiere que algunas descripciones fueron asignadas erróneamente a más de un producto.
- Los 722 valores únicos en Quantity apuntan a una gran dispersión, posiblemente indicativa de outliers o valores erróneos.
- Existen menos valores únicos en UnitPrice (1.630) que en StockCode (4.070), por lo que varios productos comparten el mismo precio.
- El dataset abarca transacciones de **38 países**, confirmando el carácter internacional del negocio.

2.5. Variables numéricas

Al observar las principales métricas de las variables numéricas se pueden apreciar varias cosas. Existen valores negativos en **Quantity** y **UnitPrice**, que pueden suponer datos erróneos. Las desviaciones estándar son muy altas, indicando una gran dispersión y posible presencia de outliers. **CustomerID** es tratado como numérico por defecto, pero es un identificador de cliente y no debe analizarse como tal; no obstante, se incluye aquí como resultado de la importación automática.

Cuadro 4: Estadísticas descriptivas de las variables numéricas del dataset.

Estadístico	Quantity	UnitPrice	CustomerID
count	541.909,00	541.909,00	406.829,00
mean	9,55	4,61	15.287,69
std	218,08	96,76	1.713,60
min	-80.995,00	-11.062,06	12.346,00
25 %	1,00	1,25	13.953,00
50 %	3,00	2,08	15.152,00
75 %	10,00	4,13	16.791,00
max	80.995,00	38.970,00	18.287,00

2.6. Variables categóricas

Se realiza un análisis descriptivo de las variables de tipo texto para obtener una visión general de la distribución de los datos.

Cuadro 5: Estadísticas descriptivas de las variables categóricas del dataset (I).

Estadístico	InvoiceNo	StockCode	Description
count	541.909	541.909	540.455
unique	25.900	4.070	4.223
top	573585	85123A	WHITE HANGING HEART T-LIGHT HOLDER
freq	1.114	2.313	2.369

Cuadro 6: Estadísticas descriptivas de las variables categóricas del dataset (II).

Estadístico	InvoiceDate	Country
count	541.909	541.909
unique	23.260	38
top	10/31/2011 14:41	United Kingdom
freq	1.114	495.478

Del análisis de los valores más y menos frecuentes se extraen las siguientes observaciones:

- Existen facturas con una gran cantidad de apariciones, lo que indica compras con muchos productos, posiblemente de grandes clientes o empresas, frente a otras que aparecen una sola vez.
- Hay productos (**StockCode**) con una demanda muy superior a otros; algunos con registros mínimos podrían tratarse de errores o productos con muy poca demanda.
- Las descripciones del top de frecuencia no siempre coinciden con los productos más vendidos, y entre los valores menos frecuentes aparecen descripciones con contenido anómalo, posiblemente errores de registro o entradas sin asignar.
- **Reino Unido** es con diferencia el país con más operaciones de compra, seguido a gran distancia de Alemania y Francia.

2.7. Análisis del identificador de factura (InvoiceNo)

Los valores de **InvoiceNo** presentan dos posibles longitudes: 6 o 7 caracteres. En todos los casos con 7 caracteres los identificadores contienen caracteres no numéricos, siguiendo siempre el patrón de una letra seguida de 6 dígitos.

Distribución de longitudes:

InvoiceNo

6 532618

7 9291

InvoiceNo con caracteres no numéricos: 9291

InvoiceNo con caracteres no numéricos y longitud 6: 0

InvoiceNo con caracteres no numéricos y longitud 7: 9291

InvoiceNo de 7 caracteres con formato letra + 6 números: 9291 / 9291

Letras únicas por las que empiezan: ['A', 'C']

Conteo de InvoiceNo por letra inicial:

InvoiceNo

A 3

C 9288

Cantidad total de valores negativos en Quantity (InvoiceNo longitud 7): 9288

Cantidad total de valores cero en Quantity (InvoiceNo longitud 7): 0

Cantidad total de valores positivos en Quantity (InvoiceNo longitud 7): 3

La gran mayoría de los InvoiceNo de 7 caracteres comienzan por la letra **C** (9.288 registros), lo que los identifica como facturas canceladas (*cancelled*). Tan solo 3 registros comienzan por la letra **A**. Como se aprecia en la tabla siguiente, estos tres casos carecen de CustomerID y presentan precios negativos, siendo identificados como ajustes de deuda incorrectos.

Cuadro 7: Facturas con InvoiceNo iniciado en “A” y Quantity positiva: identificación (I).

InvoiceNo	StockCode	Description	Qty
A563185	B	Adjust bad debt	1
A563186	B	Adjust bad debt	1
A563187	B	Adjust bad debt	1

Cuadro 8: Facturas con InvoiceNo iniciado en “A” y Quantity positiva: detalle (II).

InvoiceNo	InvoiceDate	UnitPrice	CustomerID	Country
A563185	8/12/2011 14:50	11062.06	NaN	United Kingdom
A563186	8/12/2011 14:51	-11062.06	NaN	United Kingdom
A563187	8/12/2011 14:52	-11062.06	NaN	United Kingdom

Se concluye que todos los registros con InvoiceNo de 7 caracteres serán eliminados en la fase de limpieza, ya que no representan ventas normales.

2.8. Análisis del código de producto (StockCode)

Los valores de StockCode presentan longitudes muy variables, aunque la gran mayoría se concentra en 5 o 6 caracteres.

Distribución de longitudes:

StockCode

1	715
2	144
3	710
4	1276
5	487036
6	51488
7	390
8	31
9	48
12	71

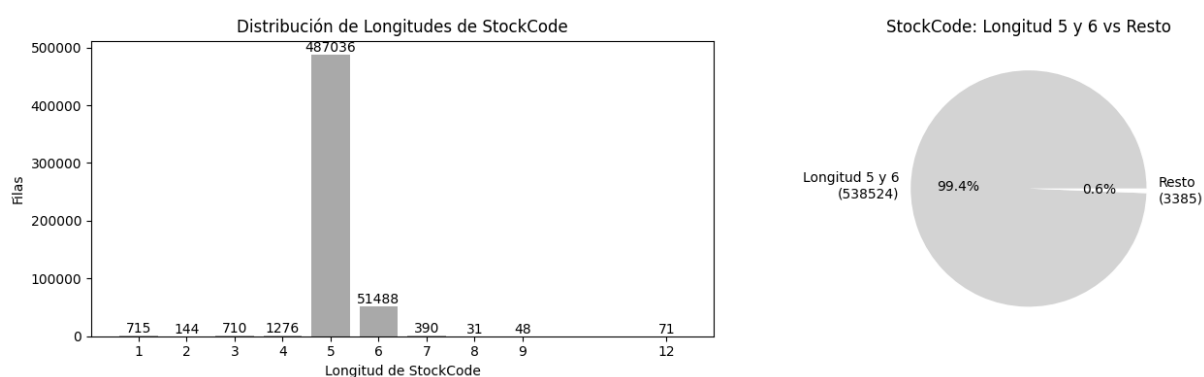


Figura 3: Distribución de longitudes de StockCode y peso relativo de las longitudes 5 y 6.

Los **StockCode** con longitudes distintas de 5 y 6 muestran descripciones que no se corresponden con productos reales (etiquetas de gestión interna, campos vacíos, etc.) y serán descartados. Para las longitudes mayoritarias, el formato observado es:

- **Longitud 5:** 5 dígitos numéricos. El 100 % de los registros cumple este formato.
- **Longitud 6:** 5 dígitos numéricos seguidos de una letra. Existe una minoría con letra minúscula.

Cuadro 9: Análisis de cumplimiento de formato en StockCodes de longitud 5 y 6.

Longitud	Total	Cumplen formato	Letra mayúscula	Letra minúscula
5 (solo dígitos)	487.036	487.036	—	—
6 (5 dígitos + 1 letra)	51.488	51.488	49.608	1.880

Los 1.880 `StockCode` de longitud 6 con letra minúscula presentan el 100 % de sus registros con `CustomerID` nulo, lo que los señala como entradas que no pertenecen a ventas normales.

Cantidad de filas con `CustomerID` nulo en `StockCodes` de longitud 6 con formato (5 números + 1 letra minúscula): 1880 | Totales: 1880

Finalmente, se comprueba si cada `StockCode` está siempre asociado a la misma descripción:

`StockCodes` con descripción única: 3308
`StockCodes` con descripciones múltiples: 650

Se detectan 650 `StockCodes` con descripciones múltiples, lo que indica inconsistencias en el registro de datos. En algunos casos las descripciones adicionales están en minúscula y no hacen referencia al producto; en otros existen variaciones menores entre descripciones en mayúsculas. Las descripciones en minúscula serán tratadas como inválidas.

2.9. Análisis de la cantidad (`Quantity`)

Los valores negativos o iguales a cero en `Quantity` no representan ventas normales, sino devoluciones, cancelaciones o datos erróneos, y serán eliminados en la fase de limpieza.

Cantidad de filas con `Quantity` negativa o cero: 10624 / 541909
Porcentaje de filas con `Quantity` ≤ 0 : 1.96%

2.10. Análisis del precio unitario (`UnitPrice`)

La variable `UnitPrice` también presenta valores negativos y valores iguales a cero. Los valores negativos no tienen sentido económico. Los valores cero se interpretarán como muestras gratuitas o regalos. Ninguno de estos casos se considerará para el análisis final.

Cantidad de filas con `UnitPrice` negativo o cero: 2517 / 541909
Porcentaje de filas con `UnitPrice` ≤ 0 : 0.46%

2.11. Análisis de la fecha de factura (`InvoiceDate`)

Dado que `InvoiceDate` está almacenada como cadena de texto, el resultado de `min()` y `max()` sobre la columna corresponde al orden lexicográfico, no cronológico. Esta comprobación se utiliza únicamente para descartar de forma rápida la presencia de fechas claramente anómalas (años fuera del rango esperado).

Rango de fechas en InvoiceDate: Desde 1/10/2011 10:04 hasta 9/9/2011 9:52

Los valores obtenidos pertenecen al rango temporal esperado del dataset. La verificación precisa del rango cronológico se realizará tras convertir la columna a tipo `datetime` en la fase de limpieza.

2.12. Análisis del identificador de cliente (CustomerID)

CustomerID es tratado como numérico (`float64`) por defecto al importar el CSV. Al convertirlo a cadena de texto, todos los valores no nulos presentan una longitud de 7 caracteres (formato `XXXXXX.0`). Deberá convertirse a tipo String en la fase de limpieza, ya que es un identificador categórico.

Distribución de longitudes en CustomerID (como string):

CustomerID

7.0 406829

2.13. Análisis del país (Country)

La variable Country contiene **38 países** distintos. **Reino Unido** concentra con diferencia el mayor número de transacciones (495.478), seguido muy por detrás por Alemania y Francia.

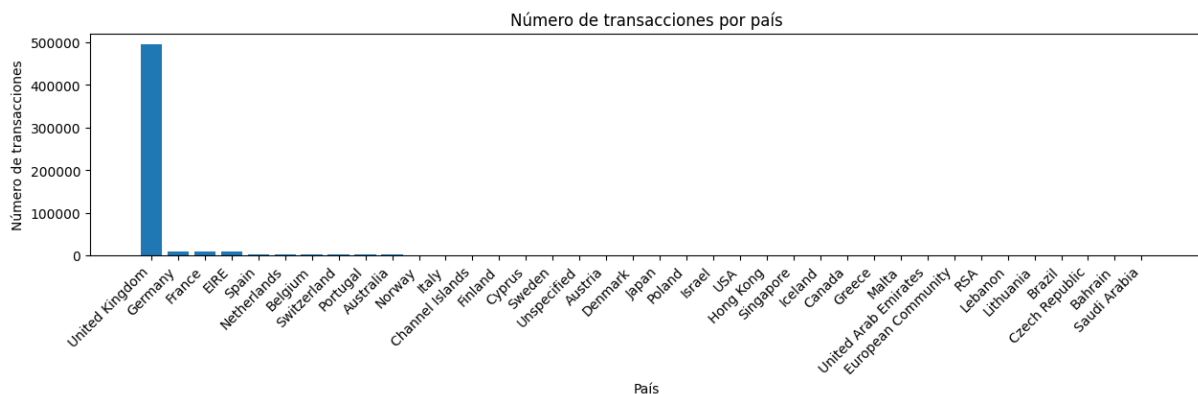


Figura 4: Número de transacciones por país.

2.14. Detección de outliers

Para la detección de outliers en las variables `Quantity` y `UnitPrice` se emplea el método del **rango intercuartílico (IQR)**, calculado entre los percentiles 20 y 80. Los valores situados por debajo de $Q_{20} - 1,5 \times IQR$ o por encima de $Q_{80} + 1,5 \times IQR$ se consideran

atípicos. Este enfoque permite identificar valores extremos que podrían distorsionar los modelos. Los outliers detectados serán eliminados durante la fase de limpieza.

2.14.1. Outliers en Quantity

Percentil 20 para Quantity: 1.0

Percentil 80 para Quantity: 12.0

Limite inferior para Quantity: -15.5

Limite superior para Quantity: 28.5

Cantidad de outliers en Quantity: 28985

Porcentaje de outliers en Quantity: 5.35%

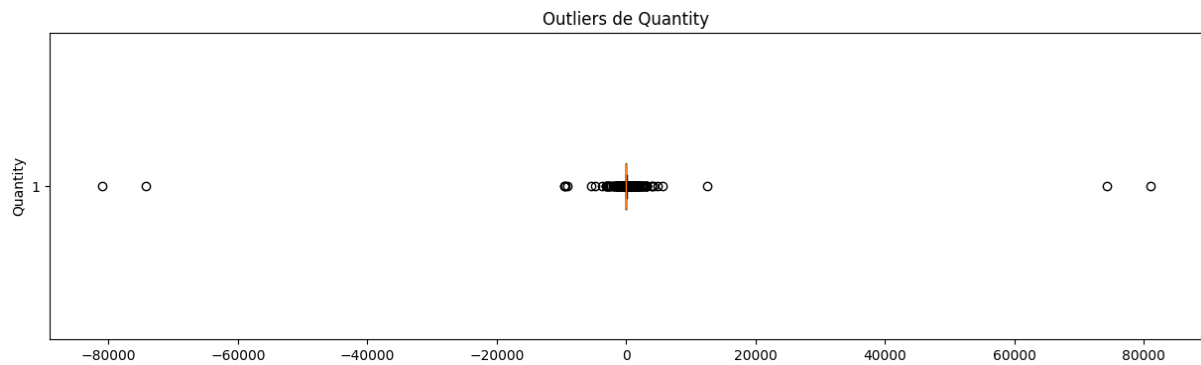


Figura 5: Diagrama de caja (boxplot) de la variable Quantity con outliers visibles.

2.14.2. Outliers en UnitPrice

Percentil 20 para UnitPrice: 0.85

Percentil 80 para UnitPrice: 4.95

Limite inferior para UnitPrice: -5.30

Limite superior para UnitPrice: 11.10

Cantidad de outliers en UnitPrice: 20141

Porcentaje de outliers en UnitPrice: 3.72%

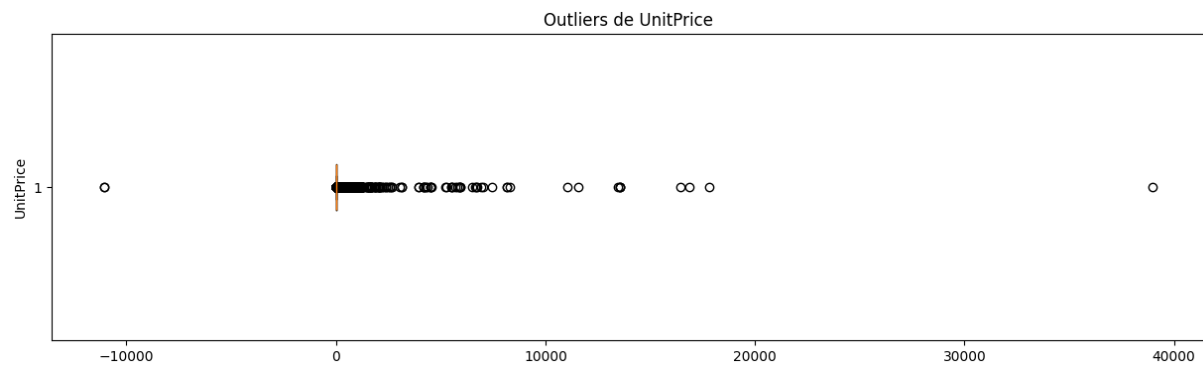


Figura 6: Diagrama de caja (boxplot) de la variable UnitPrice con outliers visibles.

3. Limpieza de Datos

Gracias a la exploración inicial se han identificado una serie de registros que reducen la calidad del dataset. En esta sección se aplican los pasos necesarios para obtener un conjunto de datos limpio y fiable sobre el que construir los modelos. Los casos tratados son los siguientes:

- Valores duplicados.
- Identificadores de factura incorrectos.
- Identificadores de producto incorrectos.
- Descripciones de producto inválidas y nulas.
- Descripciones duplicadas para un mismo producto.
- Valores negativos y ceros en `Quantity`.
- Valores negativos y ceros en `UnitPrice`.
- Valores nulos en `CustomerID`.
- Tratamiento de outliers.

Las librerías utilizadas son **Pandas** para la manipulación de datos y **Matplotlib** para la visualización. El dataset se carga aplicando la misma codificación `ISO-8859-1` utilizada durante la exploración.

3.1. Valores duplicados

Como se observó en la exploración, existen filas duplicadas que añaden peso incorrecto a los análisis. Al representar únicamente el 1 % del total, su eliminación tiene un impacto mínimo sobre el tamaño de la muestra.

Filas totales al inicio: 541909

Filas totales tras eliminar duplicados: 536641

Filas eliminadas: 5268

3.2. Identificadores de factura (InvoiceNo)

Los `InvoiceNo` de 7 caracteres corresponden siempre a facturas canceladas (prefijo **C**) o a ajustes de deuda incorrectos (prefijo **A**), tal y como se detectó durante la exploración. Ninguno de estos casos representa una venta normal, por lo que se filtran conservando únicamente los identificadores de exactamente 6 caracteres numéricos.

Filas totales tras eliminar `InvoiceNo` inválidos: 527387 (eliminadas 9254)

3.3. Identificadores de producto (StockCode)

Más del 99 % de los `StockCode` tienen longitud 5 o 6, y el resto presentaba descripciones que no corresponden a ventas normales. Se eliminan todos los registros cuyo `StockCode` no tenga longitud 5 ni 6.

Filas totales tras eliminar `StockCode` inválidos: 524599 (eliminadas 2788)

3.4. Descripciones de producto

3.4.1. Descripciones inválidas

Las descripciones en minúscula no hacen referencia al propio producto, sino a errores de gestión o registros internos. Se eliminan todas las filas cuya descripción no esté completamente en mayúsculas, así como las filas con descripción nula.

Filas totales tras eliminar descripciones inválidas: 520843 (eliminadas 3756)

Filas totales pre unificar descripciones: 520843

`StockCodes` con múltiples descripciones pre unificar: 232

Filas totales post unificar descripciones: 520843

`StockCodes` con múltiples descripciones post unificar: 0

3.4.2. Descripciones duplicadas

En la exploración se detectaron 650 `StockCode` con más de una descripción. Tras eliminar las descripciones en minúscula, quedaron 232 productos con variaciones menores entre descripciones válidas. Dado que hacen referencia al mismo artículo, se unifica la descripción de cada `StockCode` con la más frecuente, sin eliminar ninguna fila. El resultado es que ningún producto queda con dos o más descripciones diferentes.

3.5. Valores negativos y ceros en Quantity

Los valores negativos y ceros en `Quantity` no representan ventas normales. Tras las limpiezas anteriores, quedan únicamente 8 registros con `Quantity` ≤ 0 , que son eliminados.

```
Filas totales pre eliminar Quantity <= 0: 520843
```

```
Filas con Quantity <= 0: 8
```

```
Filas totales post eliminar Quantity <= 0: 520835
```

3.6. Valores negativos y ceros en UnitPrice

De igual forma, los valores negativos y ceros en `UnitPrice` carecen de sentido económico. Se eliminan los 396 registros afectados sin distinguir entre negativo y cero, ya que en ambos casos no aportan información útil.

```
Filas totales pre eliminar UnitPrice <= 0: 520835
```

```
Filas con UnitPrice <= 0: 396
```

```
Filas totales post eliminar UnitPrice <= 0: 520439
```

3.7. Nulos en CustomerID

Los identificadores de cliente nulos representaban un porcentaje significativo en el análisis inicial (24,9 %). Tras las etapas anteriores de limpieza, persisten 130.935 filas con `CustomerID` nulo. Se intenta recuperar estos valores aprovechando que, si dentro de una misma factura existen filas con cliente y filas sin cliente, se puede propagar el identificador válido.

```
Filas con CustomerID nulo pre recuperación: 130935
```

```
Facturas con CustomerID nulos y válidos (recuperables): 0
```

```
CustomerID nulos tras recuperación: 130935
```

```
CustomerID recuperados: 0
```

Sin embargo, no existe ninguna factura con mezcla de registros nulos y válidos: todas las facturas son íntegramente nulas o íntegramente válidas. Por tanto no se puede recuperar ningún valor, y los nulos permanecen en el dataset, con un tratamiento diferenciado según el modelo destino (véase la sección de exportación).

3.8. Tratamiento de outliers

Para finalizar la limpieza se aplica el mismo criterio IQR (percentiles 20–80) utilizado en la exploración, ahora sobre el dataset ya filtrado. Se eliminan los registros cuya `Quantity` o `UnitPrice` supera los límites calculados.

Outliers en Quantity: 26686 | Límites: [-15.50, 28.50]

Outliers en UnitPrice: 16920 | Límites: [-5.30, 11.10]

Filas totales pre eliminar outliers: 520439

Filas totales post eliminar outliers: 476888 (eliminadas 43551)

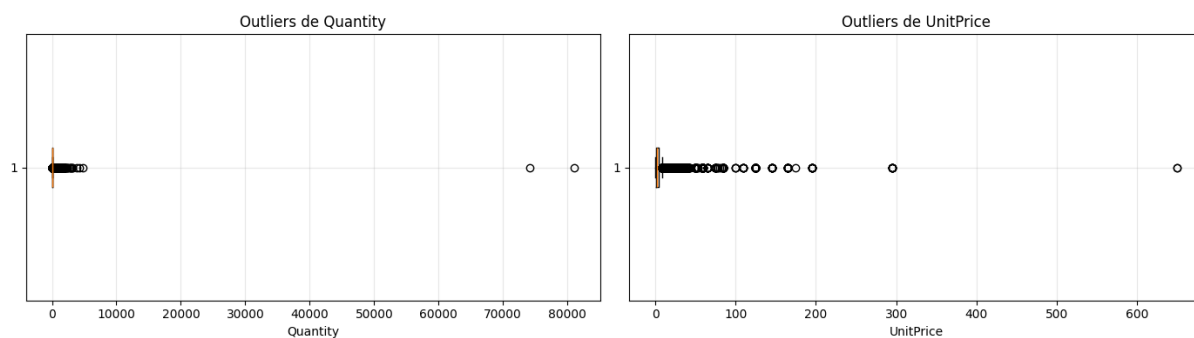


Figura 7: Diagramas de caja de Quantity y UnitPrice tras la limpieza previa a la eliminación de outliers.

3.9. Resumen de la limpieza

La tabla siguiente recoge el número de filas eliminadas en cada etapa y el tamaño resultante del dataset.

Cuadro 10: Resumen de filas eliminadas en cada etapa de limpieza.

Etapas	Filas antes	Eliminadas	Filas después
Valores duplicados	541.909	5.268	536.641
InvoiceNo inválidos	536.641	9.254	527.387
StockCode inválidos	527.387	2.788	524.599
Descripciones inválidas	524.599	3.756	520.843
Descripciones duplicadas	520.843	0	520.843
Quantity ≤ 0	520.843	8	520.835
UnitPrice ≤ 0	520.835	396	520.439
Outliers (Qty + Price)	520.439	43.551	476.888
Total eliminadas	541.909	65.021	476.888

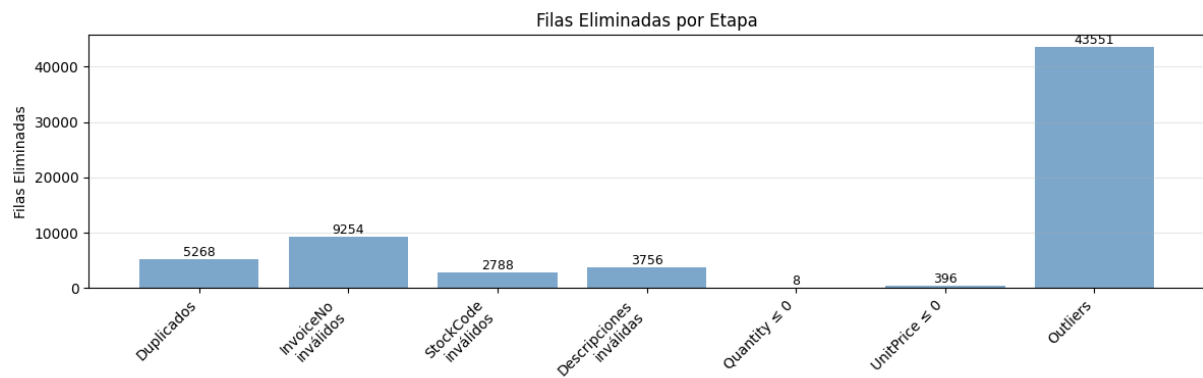


Figura 8: Filas eliminadas en cada etapa del proceso de limpieza.

El resumen de las principales métricas del dataset limpio es el siguiente:

Cuadro 11: Estadísticas del dataset tras la limpieza completa.

Dimensiones		Cardinalidades	
Filas totales	476.888	Facturas únicas	18.338
Columnas	8	Clientes únicos	4.208
CustomerID nulos	120.892	Productos únicos	3.774
Resto de nulos	0	Países únicos	38
Rangos de variables numéricas			
Quantity mínimo	1	UnitPrice mínimo	0,06
Quantity máximo	28	UnitPrice máximo	11,05
Quantity media	6,06	UnitPrice media	2,87

Filas finales tras limpieza: 476888

Valores nulos:

InvoiceNo	0
StockCode	0
Description	0
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	120892
Country	0

Facturas únicas: 18338

Clientes únicos: 4208

Productos únicos: 3774

Países únicos: 38

Quantity → mínimo: 1 máximo: 28 media: 6.06

UnitPrice → mínimo: 0.06 máximo: 11.05 media: 2.87

3.10. Exportación del dataset limpio

Una vez completada la limpieza, el dataset se exporta en dos versiones según el modelo destino.

3.10.1. Dataset para regresión

Para el modelo de regresión, cuyo objetivo es predecir ventas diarias, la variable `CustomerID` no es relevante. Por tanto, se conservan todas las filas del dataset limpio, incluyendo las que tienen `CustomerID` nulo.

3.10.2. Dataset para clustering

El modelo de clustering segmenta clientes, por lo que requiere un `CustomerID` válido en todos los registros. Las 120.892 filas con `CustomerID` nulo se eliminan en esta versión.

Dataset de regresión exportado: 476888 filas

Dataset de clustering exportado: 355996 filas

Filas eliminadas por `CustomerID` nulo: 120892

4. Tratamiento de datos para Clustering

Para el entrenamiento de los modelos de clustering se necesita obtener datos representativos de los clientes para poder agruparlos según sus características de compra. Las tareas realizadas en esta fase son las siguientes:

- Cargar el CSV limpio para clustering.
- Prescindir de las variables innecesarias.
- Correcciones en el tipado de las columnas.
- Generación de nuevas variables a partir de las características de los clientes.
- Eliminación de valores extremos.
- Normalización de los datos.

Las librerías utilizadas son **Pandas** para la manipulación de datos, **Matplotlib** para la visualización y **MinMaxScaler** de *scikit-learn* para la normalización.

4.1. Carga y verificación del dataset

Se carga el CSV limpio para clustering generado en la fase anterior. El dataset contiene 355.996 filas y 8 columnas, sin ningún valor nulo (los registros con **CustomerID** nulo ya fueron eliminados).

Filas: 355996, Columnas: 8

InvoiceNo	int64
StockCode	str
Description	str
Quantity	int64
InvoiceDate	str
UnitPrice	float64
CustomerID	float64
Country	str

Valores nulos: ninguno en ninguna columna.

4.2. Selección de variables

El objetivo es obtener características de los clientes para poder agruparlos. Las variables seleccionadas son:

- **InvoiceNo**: para extraer el número de facturas de cada cliente.
- **StockCode**: para conocer la variedad de productos comprados.
- **Quantity**: para conocer la cantidad total de artículos adquiridos.
- **InvoiceDate**: para calcular la frecuencia de compra.
- **UnitPrice**: combinado con la cantidad, refleja el poder adquisitivo del cliente.
- **CustomerID**: identificador de cada cliente.

Se descartan **Description** y **Country**. La descripción no aporta información relevante sobre el comportamiento del cliente. En cuanto al país, el dominio absoluto del Reino Unido (más del 90 % de las transacciones) podría generar inconsistencias en la segmentación geográfica.

4.3. Corrección del tipado

Varias columnas se encontraban con tipos incorrectos tras la importación del CSV. Se aplican las siguientes conversiones:

- **InvoiceDate**: de cadena de texto a `datetime`, permitiendo cálculos temporales.
- **InvoiceNo**: de entero a `string`, ya que es un identificador categórico.
- **CustomerID**: de `float64` a `string` por el mismo motivo.

<code>InvoiceNo</code>	<code>string</code>
<code>StockCode</code>	<code>str</code>
<code>Quantity</code>	<code>int64</code>
<code>InvoiceDate</code>	<code>datetime64[us]</code>
<code>UnitPrice</code>	<code>float64</code>
<code>CustomerID</code>	<code>string</code>

4.4. Generación de nuevas variables

El siguiente paso consiste en agregar los datos por cliente y derivar variables que resuman su comportamiento. El período de análisis abarca 373 días (del 1 de diciembre de 2010 al 9 de diciembre de 2011). Las variables generadas son:

- **Transacciones**: número de facturas únicas por cliente (**InvoiceNo** únicos).
- **ProductosDistintos**: número de productos diferentes comprados por cliente (**StockCode** únicos).
- **GastoTotal**: gasto acumulado del cliente, calculado como $\sum(\text{Quantity} \times \text{UnitPrice})$.
- **FrecuenciaCompra**: media de transacciones por día, calculada como $\text{Transacciones} / \text{días totales}$.

La agrupación por **CustomerID** produce un dataset de 4.208 registros, uno por cliente:

Cuadro 12: Primeras 5 filas del dataset agrupado por cliente (variables originales).

CustomerID	InvoiceNo	StockCode	Quantity	UnitPrice	InvoiceDate
12347	7	98	1909	2,473	7
12348	3	6	140	0,650	3
12349	1	66	551	3,355	1
12350	1	16	196	1,581	1
12352	7	56	521	3,844	7

Tras añadir las variables derivadas y eliminar las columnas originales que ya no son necesarias (**InvoiceNo**, **StockCode**, **Quantity**, **UnitPrice**, **InvoiceDate**), el dataset queda con 5 columnas:

Cuadro 13: Primeras 5 filas del dataset de clientes con las variables derivadas (antes de normalizar).

CustomerID	Transacciones	ProductosDistintos	GastoTotal	FrecuenciaCompra
12347	7	98	4721,44	0,018767
12348	3	6	91,00	0,008043
12349	1	66	1848,77	0,002681
12350	1	16	309,93	0,002681
12352	7	56	2002,79	0,018767

4.5. Eliminación de valores extremos

La agregación por cliente puede producir valores muy alejados de los típicos. Se aplica el método IQR estándar (percentiles 25 y 75) sobre las cuatro variables numéricas generadas.

Las estadísticas descriptivas previas a la eliminación revelan rangos muy amplios:

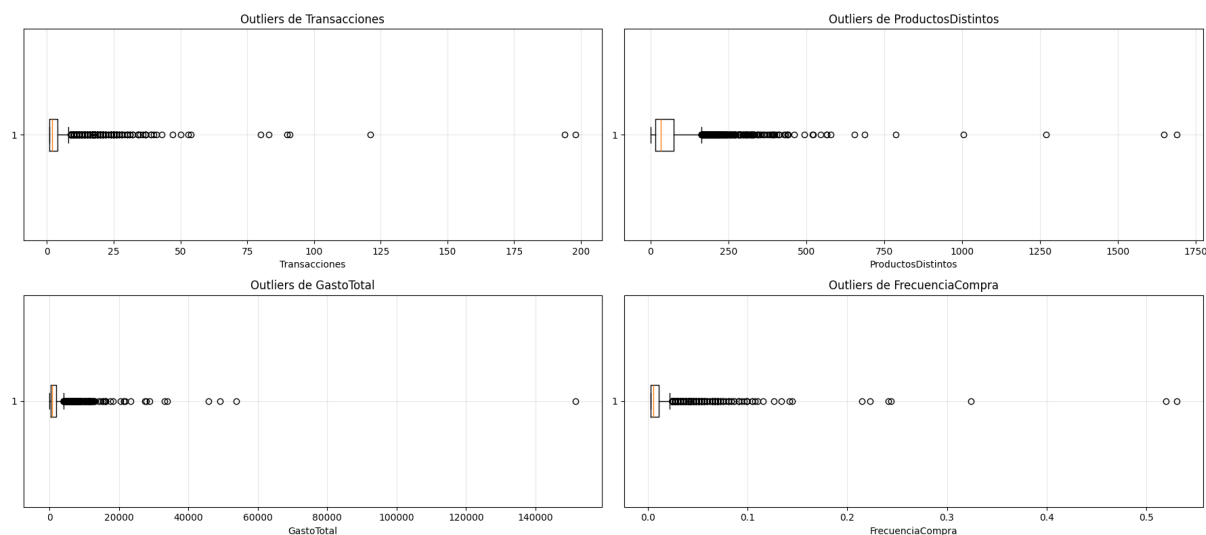


Figura 9: Diagramas de caja de las cuatro variables de clustering antes de eliminar outliers.

Tras la eliminación, el dataset se reduce a 3.592 clientes y las distribuciones quedan notablemente más compactas:

Cuadro 14: Estadísticas descriptivas de clustering tras eliminar outliers: volumen (I).

Estadístico	Transacciones	ProdDistintos	GastoTotal
count	3.592	3.592	3.592
mean	2,46	38,41	870,93
std	1,76	33,79	823,73
min	1,00	1,00	1,90
25 %	1,00	13,00	271,68
50 %	2,00	28,00	584,35
75 %	3,00	54,00	1.200,46
max	8,00	163,00	4.016,67

Cuadro 15: Estadísticas descriptivas de clustering tras eliminar outliers: frecuencia (II).

Estadístico	FrecuenciaCompra
count	3.592
mean	0,006582
std	0,004709
min	0,002681
25 %	0,002681
50 %	0,005362
75 %	0,008043
max	0,021448

4.6. Normalización

Se aplica **MinMaxScaler** para escalar las cuatro variables al rango $[0, 1]$. Este método es sensible a los valores extremos, por lo que es imprescindible haberlos eliminado previamente para obtener una normalización de calidad. El resultado es una distribución homogénea que permite que ninguna variable domine al resto durante el entrenamiento de los modelos.

Cuadro 16: Estadísticas descriptivas de las variables de clustering tras la normalización MinMaxScaler $[0, 1]$.

Estadístico	Transacciones	ProdDistintos	GastoTotal	FrecuenciaCompra
count	3.592	3.592	3.592	3.592
mean	0,2079	0,2309	0,2165	0,2079
std	0,2509	0,2086	0,2052	0,2509
min	0,0000	0,0000	0,0000	0,0000
25 %	0,0000	0,0741	0,0672	0,0000
50 %	0,1429	0,1667	0,1451	0,1429
75 %	0,2857	0,3272	0,2985	0,2857
max	1,0000	1,0000	1,0000	1,0000

4.7. Exportación del dataset transformado

El dataset final, compuesto por 3.592 clientes y las columnas **CustomerID**, **Transacciones**, **ProductosDistintos**, **GastoTotal** y **FrecuenciaCompra** (todas normalizadas), se exporta para su uso en los modelos de clustering.

Dataset de clustering exportado: 3592 filas

Columnas: ['CustomerID', 'Transacciones', 'ProductosDistintos',
 'GastoTotal', 'FrecuenciaCompra']

Archivo: ../output/clustering/data_transformado_clustering.csv

5. Modelo K-Means

El objetivo de este modelo es la segmentación de clientes de e-commerce según su comportamiento de compra utilizando el algoritmo **K-Means**. Se parte del dataset transformado generado en la fase de tratamiento, compuesto por 3.592 clientes con cuatro variables normalizadas: *Transacciones*, *ProductosDistintos*, *GastoTotal* y *FrecuenciaCompra*.

Las librerías utilizadas son **Pandas** para la manipulación de datos, **Matplotlib** para la visualización, y **Sklearn** para la implementación del modelo, la evaluación de su rendimiento y la reducción de dimensionalidad mediante PCA.

5.1. Carga de datos

Se carga el CSV previamente transformado con las características clave de cada cliente:

Shape: (3592, 5)

	CustomerID	Transacciones	ProductosDistintos	GastoTotal	FrecuenciaCompra
0	12348	0.285714	0.030864	0.022193	0.285714
1	12349	0.000000	0.401235	0.460020	0.000000
2	12350	0.000000	0.092593	0.076723	0.000000
3	12352	0.857143	0.339506	0.498384	0.857143
4	12353	0.000000	0.018519	0.029790	0.000000

Shape escalado: (3592, 4)

5.2. Preparación de los datos

Se eliminan la columna **CustomerID**, que no aporta información para la segmentación, y se escala la matriz de características con **StandardScaler**. Este reescalado es necesario porque K-Means utiliza distancias euclídeas y es sensible a la magnitud de las variables; al aplicar el escalado se consigue que todas las dimensiones tengan igual peso en el cálculo de los centroides.

5.3. Número óptimo de clusters

Para determinar el valor de K más adecuado se aplican dos criterios complementarios: el **método del codo** (inercia) y el **Silhouette Score**. Se evalúan valores de K entre 2 y 10:

Cuadro 17: Inercia y Silhouette Score para K de 2 a 10 en K-Means.

K	Inercia	Silhouette Score
2	6.371,72	0,5516
3	4.718,78	0,4529
4	3.842,84	0,4601
5	3.264,38	0,3861
6	2.862,01	0,3937
7	2.502,77	0,3887
8	2.248,51	0,3885
9	2.071,08	0,3831
10	1.919,65	0,3936

k= 2: Inercia=6371.72, Silhouette=0.5516

k= 3: Inercia=4718.78, Silhouette=0.4529

k= 4: Inercia=3842.84, Silhouette=0.4601 <-- seleccionado

k= 5: Inercia=3264.38, Silhouette=0.3861

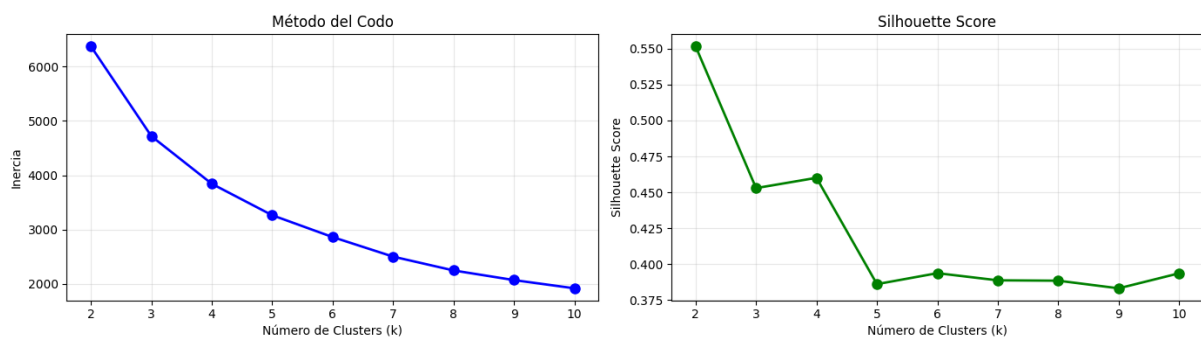
k= 6: Inercia=2862.01, Silhouette=0.3937

k= 7: Inercia=2502.77, Silhouette=0.3887

k= 8: Inercia=2248.51, Silhouette=0.3885

k= 9: Inercia=2071.08, Silhouette=0.3831

k=10: Inercia=1919.65, Silhouette=0.3936

Figura 10: Método del codo (inercia) y Silhouette Score para K de 2 a 10.

Los valores más altos de Silhouette Score se obtienen para $K = 2$ (0,5516) y $K = 4$ (0,4601). Se elige $K = 4$ porque el codo de la inercia se aprecia claramente en ese punto y porque proporciona una segmentación con mayor riqueza interpretativa que dos únicos grupos.

5.4. Entrenamiento del modelo

Se entrena K-Means con $K = 4$, semilla `random_state=42` y `n_init=10` para garantizar la reproducibilidad y reducir la sensibilidad a la inicialización aleatoria de los centroides. La distribución resultante de clientes por cluster es la siguiente:

Modelo entrenado con K=4 clusters

Cluster	
0	417
1	2054
2	615
3	506

El Cluster 1 concentra la mayoría de los clientes (2.054), lo que indica que el segmento de compradores ocasionales es el más numeroso. Los demás clusters recogen grupos más pequeños pero con perfiles bien diferenciados.

5.5. Evaluación del modelo

Silhouette Score:	0.4601
Davies-Bouldin Index:	1.0322

El **Silhouette Score de 0,46** indica una separación razonable entre los clusters: los puntos están notablemente más cerca de su propio cluster que de los vecinos. El **Davies-Bouldin Index de 1,03** refleja que los clusters son compactos y presentan una buena separación entre sí. Ambas métricas confirman que la segmentación con $K = 4$ es sólida para este conjunto de datos.

5.6. Perfiles de clusters

A continuación se muestran las medias de las cuatro variables normalizadas para cada cluster. Estos valores permiten caracterizar el comportamiento de compra de cada segmento:

Cuadro 18: Perfiles medios de los 4 clusters K-Means: volumen de actividad (I).

Cluster	Transacciones	ProdDistintos	Cientes
0 — Muy activos	0,7153	0,5086	417
1 — Ocasionales	0,0430	0,1175	2054
2 — Activos	0,3984	0,1930	615
3 — Diversificados	0,2276	0,5088	506

Cuadro 19: Perfiles medios de los 4 clusters K-Means: poder adquisitivo (II).

Cluster	GastoTotal	FrecuenciaCompra
0 — Muy activos	0,5950	0,7153
1 — Ocasionales	0,1002	0,0430
2 — Activos	0,2234	0,3984
3 — Diversificados	0,3680	0,2276

Perfiles de clusters (media de variables normalizadas):

Cluster	Transacciones	ProductosDistintos	GastoTotal	FrecuenciaCompra
0	0.7153	0.5086	0.5950	0.7153
1	0.0430	0.1175	0.1002	0.0430
2	0.3984	0.1930	0.2234	0.3984
3	0.2276	0.5088	0.3680	0.2276

5.7. Visualización de clusters

Aplicando una reducción de dimensionalidad con **PCA** se visualizan los clusters en dos dimensiones. Las dos componentes principales capturan el 93,0 % de la varianza (PC1: 76,7 %, PC2: 16,4 %), lo que garantiza que la representación bidimensional conserva la mayor parte de la información relevante.

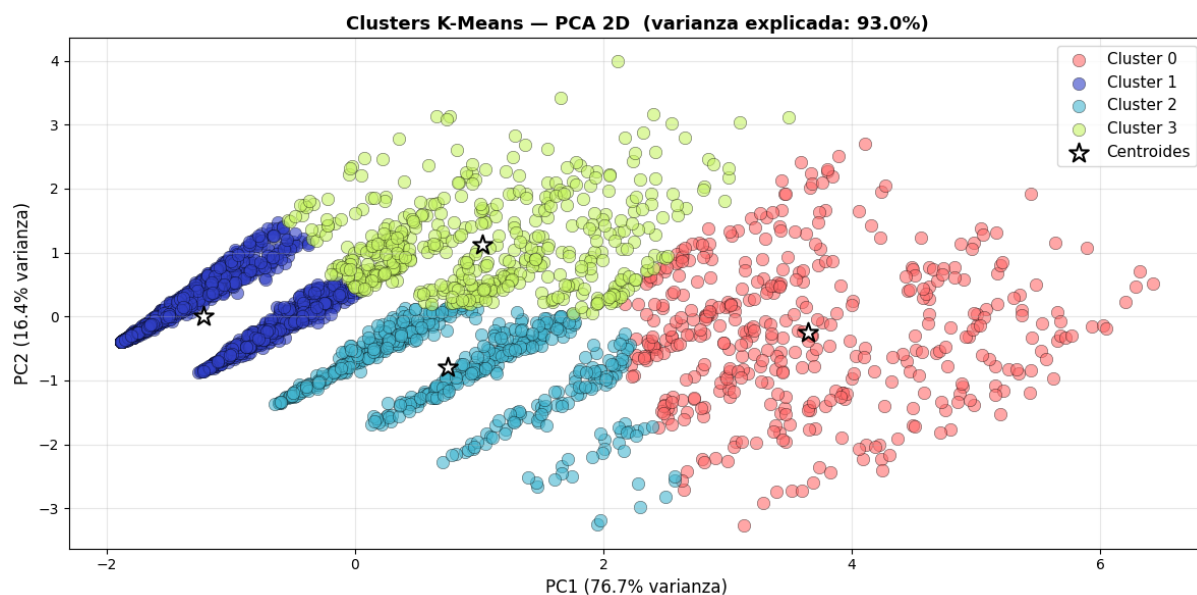


Figura 11: Clusters K-Means proyectados en el espacio PCA 2D (varianza explicada: 93,0 %). Los asteriscos indican los centroides.

Los clusters están bien diferenciados en el espacio PCA, con mayor dispersión en el Cluster 0, que agrupa a los clientes más activos y con mayor heterogeneidad de comportamiento.

5.8. Segmentación de clientes

El análisis identifica cuatro perfiles de clientes diferenciados:

Cluster 1 — Clientes Ocasionales (2.054 clientes)

Transacciones bajas, pocos productos distintos, bajo gasto y baja frecuencia. Son clientes de una única compra o muy esporádicos, sin continuidad y con escaso gasto. Representan el segmento mayoritario y el de menor valor para el negocio.

Cluster 2 — Clientes Activos (615 clientes)

Transacciones y frecuencia medias, diversidad de producto baja-media y gasto moderado. Clientes con cierta regularidad de compra, en una fase intermedia de fidelización.

Cluster 3 — Clientes Diversificados (506 clientes)

Transacciones y frecuencia moderadas, pero con una alta diversidad de productos y gasto medio-alto. Son clientes que, aunque no compran con mucha frecuencia, adquieren una gran variedad de artículos en cada pedido.

Cluster 0 — Clientes Muy Activos (417 clientes)

Transacciones y frecuencia muy altas, alta diversidad de producto y gasto elevado. Son los clientes más valiosos: compran con mucha frecuencia, adquieren gran variedad de productos y tienen un alto volumen de gasto total.

6. Modelo DBSCAN

El objetivo de este modelo es la segmentación de clientes de e-commerce según su comportamiento de compra utilizando el algoritmo **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*). A diferencia de K-Means, DBSCAN no requiere especificar el número de clusters de antemano y es capaz de detectar automáticamente puntos de ruido que no pertenecen a ningún grupo.

Las librerías utilizadas son **Pandas** para la manipulación de datos, **Matplotlib** para la visualización, y **Scikit-learn** para la implementación del algoritmo DBSCAN, la evaluación de métricas y la reducción de dimensionalidad con PCA.

6.1. Carga de datos

Se carga el CSV previamente transformado con las características clave de cada cliente:

Shape: (3592, 5)

	CustomerID	Transacciones	ProductosDistintos	GastoTotal	FrecuenciaCompra
0	12348	0.285714	0.030864	0.022193	0.285714
1	12349	0.000000	0.401235	0.460020	0.000000
2	12350	0.000000	0.092593	0.076723	0.000000
3	12352	0.857143	0.339506	0.498384	0.857143
4	12353	0.000000	0.018519	0.029790	0.000000

Shape escalado: (3592, 4)

6.2. Preparación de los datos

Se elimina la columna `CustomerID` y se estandarizan los datos con **StandardScaler**. DBSCAN utiliza distancias para determinar la densidad de los puntos, por lo que el escalado es imprescindible para que todas las variables tengan el mismo peso.

6.3. Búsqueda de parámetros óptimos

DBSCAN requiere dos parámetros: **eps** (radio de vecindad) y **min_samples** (puntos mínimos para formar un cluster). Para estimar el valor óptimo de eps se emplea la **curva de k-distancias**: se calcula la distancia de cada punto a su k -ésimo vecino más cercano (con $k = \text{min_samples}$), se ordenan de menor a mayor y se busca el punto de inflexión de la curva.

Percentiles de la 8-distancia (min_samples=8):

Percentil 90: 0.475

Percentil 95: 0.635

Percentil 99: 0.924

El punto de inflexión se sitúa alrededor de $\text{eps}=0.75$.

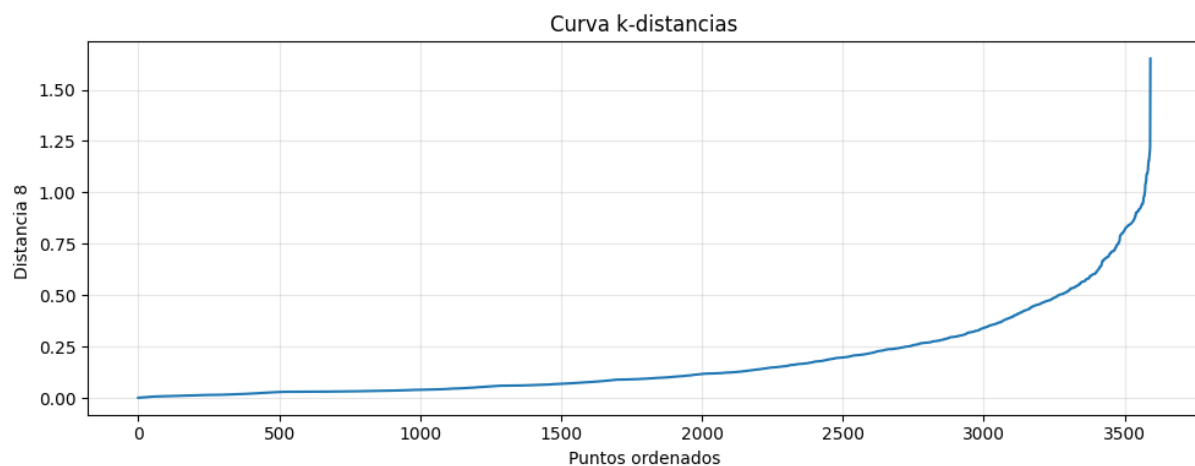


Figura 12: Curva de 8-distancias ordenadas. El punto de inflexión indica el eps óptimo ($\approx 0,75$).

El percentil 95 de la distribución de distancias se sitúa en 0,635 y el percentil 99 en 0,924. El punto de inflexión visual de la curva apunta a un valor de eps en torno a 0,75, que es el seleccionado.

6.4. Entrenamiento del modelo

Tras probar distintas combinaciones, los parámetros $\text{eps}=0.75$ y $\text{min_samples}=8$ ofrecen el mejor equilibrio entre número de clusters y porcentaje de ruido:

Parametros: $\text{eps}=0.75$, $\text{min_samples}=8$

Clusters encontrados: 8

Puntos de ruido (outliers): 46 (1.3%)

Cluster

-1	46	(ruido)
0	459	
1	1475	
2	73	

3	820
4	338
5	195
6	134
7	52

Se obtienen **8 clusters** con solo 46 puntos de ruido (1,3 %), un resultado excelente que indica que el radio de vecindad elegido cubre bien la densidad del dataset.

6.5. Evaluación del modelo

Silhouette Score: 0.2523
Davies-Bouldin Index: 2.5660

El **Silhouette Score de 0,25** es más bajo que el obtenido con K-Means (0,46). Esto es esperable en DBSCAN: el algoritmo genera clusters de densidad variable con fronteras difusas, lo que reduce la cohesión interna medida por esta métrica. El **Davies-Bouldin Index de 2,57** también refleja mayor solapamiento entre clusters adyacentes, consecuencia natural de una segmentación gradual basada en densidad. Ambos valores son aceptables para un dataset con esta estructura y deben interpretarse junto con el análisis visual y de perfiles.

6.6. Visualización de clusters

Mediante una reducción de dimensionalidad con **PCA** se visualizan los clusters en dos dimensiones. Las dos componentes principales capturan el 93,0 % de la varianza (PC1: 76,7 %, PC2: 16,4 %).

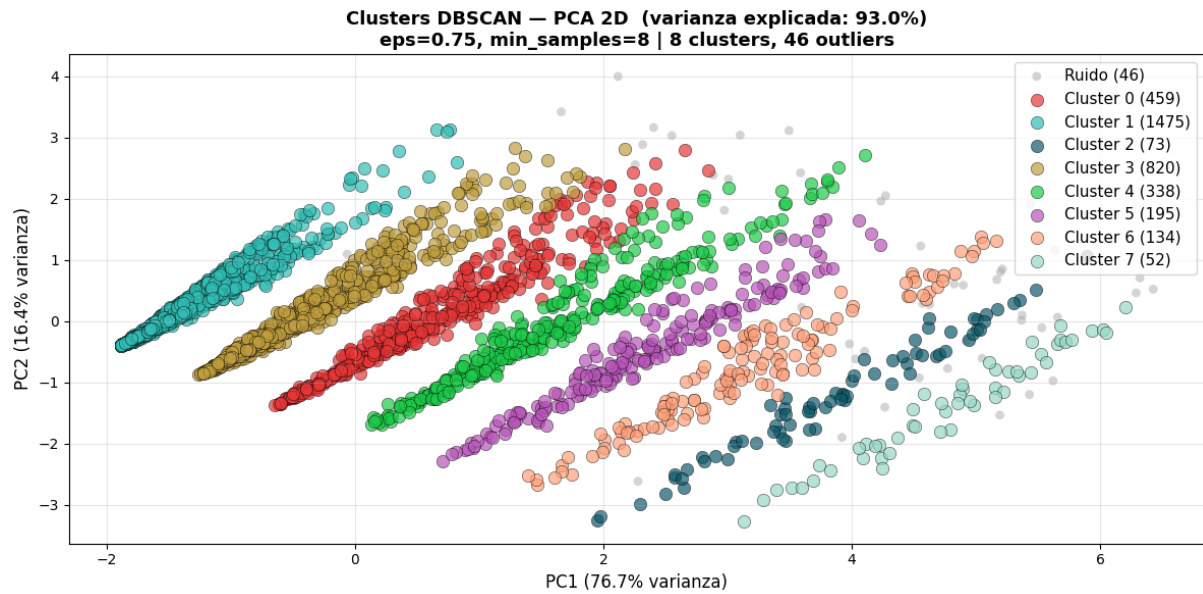


Figura 13: Clusters DBSCAN proyectados en el espacio PCA 2D (varianza explicada: 93,0 %). Los puntos grises son ruido.

Los clusters aparecen bien diferenciados en el espacio PCA. Los puntos de ruido (gris) se sitúan en los márgenes, alejados de la densidad principal, lo que confirma que DBSCAN los ha descartado correctamente.

6.7. Perfiles de clusters

A continuación se muestran las medias de las cuatro variables normalizadas para cada cluster (excluyendo el ruido). Los 8 clusters se distribuyen en 6 niveles de actividad bien diferenciados:

Cuadro 20: Perfiles medios de los 8 clusters DBSCAN (valores normalizados en $[0, 1]$, sin puntos de ruido).

Cluster	Segmento	Transacc.	ProdDist.	Gasto	Frecuencia	Clientes
1	Ocasionales	0,0000	0,1199	0,0910	0,0000	1.475
3	Ocasionales	0,1429	0,2165	0,1821	0,1429	820
0	Moderados	0,2857	0,2950	0,2632	0,2857	459
4	Activos	0,4286	0,3682	0,3616	0,4286	338
5	Muy Activos	0,5714	0,3746	0,4264	0,5714	195
6	Poderosos	0,7143	0,4060	0,4958	0,7143	134
2	Premium	0,8571	0,3983	0,5427	0,8571	73
7	Premium	1,0000	0,4601	0,6094	1,0000	52

Perfiles de clusters (media de variables normalizadas, sin ruido):

Cluster	Transacciones	ProductosDistintos	GastoTotal	FrecuenciaCompra
0	0.2857	0.2950	0.2632	0.2857
1	0.0000	0.1199	0.0910	0.0000
2	0.8571	0.3983	0.5427	0.8571
3	0.1429	0.2165	0.1821	0.1429
4	0.4286	0.3682	0.3616	0.4286
5	0.5714	0.3746	0.4264	0.5714
6	0.7143	0.4060	0.4958	0.7143
7	1.0000	0.4601	0.6094	1.0000

6.8. Segmentación de clientes

Aunque DBSCAN genera 8 clusters, estos pueden agruparse en 6 segmentos de negocio según su nivel de actividad:

Clusters 1 + 3 — Clientes Ocasionales (2.295 clientes)

Los dos clusters de menor actividad. Las transacciones y la frecuencia de compra son prácticamente nulas en el Cluster 1 y muy bajas en el Cluster 3. Son compradores esporádicos o de una única compra con muy bajo compromiso con la marca.

Cluster 0 — Clientes Moderados (459 clientes)

Actividad baja-media en todas las dimensiones. Son clientes con cierta recurrencia pero en una fase inicial de fidelización, aún con poco gasto y diversidad de producto.

Cluster 4 — Clientes Activos (338 clientes)

Nivel medio en transacciones, productos y gasto. Muestran una intención de compra consistente y buena diversidad de productos, representando un segmento de crecimiento potencial.

Cluster 5 — Clientes Muy Activos (195 clientes)

Nivel medio-alto en todas las variables. Son clientes frecuentes con fuerte compromiso y diversidad de compras, próximos al perfil de alto valor.

Cluster 6 — Clientes Poderosos (134 clientes)

Alto nivel de transacciones, productos distintos y gasto. Clientes leales y de alto valor que realizan compras frecuentes con máximo compromiso.

Clusters 2 + 7 — Clientes Premium (125 clientes)

Los dos clusters de mayor actividad. El Cluster 7 alcanza el valor máximo normalizado en transacciones y frecuencia (1,0000), representando a los clientes con la mayor lealtad y gasto del dataset. Son los *top-tier customers*, pilares del negocio.

7. Tratamiento de datos para Regresión

Para el entrenamiento de los modelos de regresión se necesita obtener variables representativas de las ventas diarias. El objetivo es predecir las **ganancias totales de cada día** a partir de patrones temporales. Las tareas realizadas en esta fase son las siguientes:

- Cargar el CSV limpio para regresión.
- Seleccionar las variables necesarias.
- Tipar correctamente la columna de fecha.
- Generar nuevas variables que capturen los patrones de venta diaria.
- Eliminar valores extremos.
- Preprocesar variables categóricas con `OneHotEncoding`.
- Normalizar las variables numéricas.
- Dividir en conjuntos de entrenamiento, validación y test.

Las librerías utilizadas son **Pandas** para la manipulación de datos, y **Sklearn** para el preprocesado (`OneHotEncoder`, `StandardScaler`).

7.1. Carga y verificación del dataset

Se carga el CSV limpio para regresión generado en la fase de limpieza. El dataset contiene 476.888 transacciones individuales y 8 columnas:

Filas: 476888, Columnas: 8

InvoiceNo	int64
StockCode	str
Description	str
Quantity	int64
InvoiceDate	str
UnitPrice	float64
CustomerID	float64
Country	str

7.2. Selección de variables

Para predecir las ventas diarias solo son necesarias tres columnas:

- **InvoiceDate**: para agrupar las transacciones por día.
- **Quantity**: número de unidades vendidas, necesario para calcular las ganancias.
- **UnitPrice**: precio unitario, necesario para calcular las ganancias.

El resto de columnas (**InvoiceNo**, **StockCode**, **Description**, **CustomerID**, **Country**) no aportan información sobre la tendencia temporal de las ventas y se eliminan.

7.3. Corrección del tipado

La columna **InvoiceDate** se encontraba como cadena de texto. Se convierte a **datetime** para poder extraer componentes temporales y realizar la agrupación por fecha:

InvoiceDate	datetime64[us]
Quantity	int64
UnitPrice	float64

7.4. Generación de nuevas variables

Esta es la fase más relevante de la preparación. El objetivo es construir un dataset con una fila por día y variables que capturen los patrones de venta:

GananciasTotales

Es la variable objetivo. Se calcula como $\text{Quantity} \times \text{UnitPrice}$ por transacción y se agrupa sumando todas las ganancias del mismo día. El dataset original tiene múltiples transacciones por día; tras la agrupación se obtienen **305 días únicos**.

DiaSemana y DiaMes

El día de la semana y el día del mes permiten al modelo capturar ciclos de compra semanales y mensuales. Como el dataset solo abarca un año, el día del año sería redundante, pero estas dos variables sí se repiten con suficiente frecuencia para aprender patrones.

Lag_1_Ventas

Las ganancias del día inmediatamente anterior. Permite al modelo capturar la tendencia reciente: si el día anterior las ventas fueron altas, es probable que el siguiente también lo sea. El primer día pierde su lag y se elimina, quedando **304 días**.

El dataset diario resultante, con las cuatro variables y la fecha, tiene el siguiente aspecto:

Cuadro 21: Primeras 5 filas del dataset diario con las variables derivadas (antes de normalizar).

InvoiceDate	GananciasTotales (€)	DiaSemana	DiaMes	Lag_1_Ventas (€)
2010-12-02	26.292,13	Jueves	2	31.052,61
2010-12-03	25.490,33	Viernes	3	26.292,13
2010-12-05	23.445,96	Domingo	5	25.490,33
2010-12-06	36.394,80	Lunes	6	23.445,96
2010-12-07	32.279,50	Martes	7	36.394,80

Las estadísticas descriptivas antes de eliminar outliers muestran una media de 19.139 €/día con una desviación notable ($\text{std} = 8.591$ €):

Cuadro 22: Estadísticas descriptivas de las variables numéricas del dataset diario (304 días, antes de eliminar outliers).

Estadístico	GananciasTotales (€)	Lag_1_Ventas (€)	DiaMes
count	304,00	304,00	304,00
mean	19.139,68	19.178,71	15,18
std	8.591,42	8.618,54	8,63
min	2.728,96	2.728,96	1,00
25 %	13.532,80	13.532,80	8,00
50 %	17.719,15	17.719,15	15,00
75 %	23.207,16	23.372,84	22,00
max	52.609,46	52.609,46	31,00

7.5. Eliminación de outliers

Se aplica el método IQR sobre **GananciasTotales**. Se detectan y eliminan **12 días atípicos** (3,95%), todos concentrados en octubre-diciembre 2011, coincidiendo con el período de Black Friday y Navidad:

Q1 (25%): 13.532,81 €
 Q3 (75%): 23.207,15 €
 IQR: 9.674,35 €
 Limite inferior: -978,72 €
 Limite superior: 37.718,68 €

Outliers detectados: 12 (3,95%)

Fecha	GananciasTotales
2011-10-31	37.954,91
2011-11-08	41.512,22
2011-11-10	42.215,80
2011-11-11	39.007,11
2011-11-16	42.380,62
2011-11-17	41.320,85
2011-11-22	39.472,09
2011-11-23	44.932,10
2011-11-29	41.559,76
2011-11-30	39.347,98
2011-12-05	52.609,46
2011-12-08	44.845,31

Filas antes: 304

Filas despues: 292

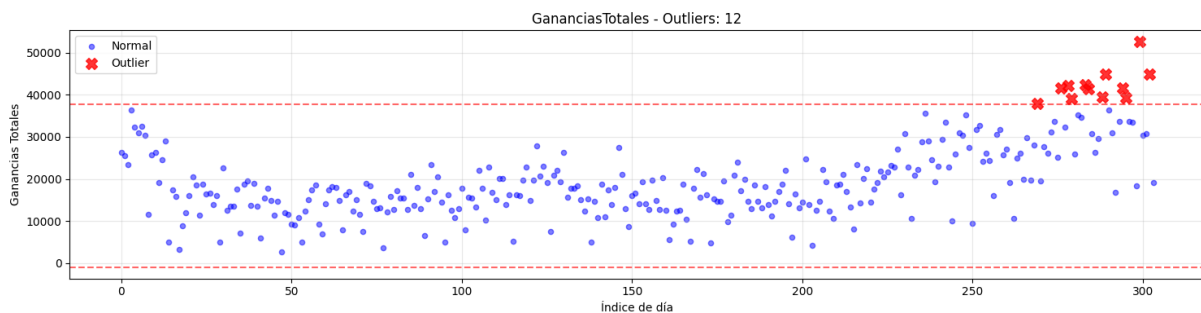


Figura 14: Distribución de GananciasTotales antes y después de eliminar outliers. Los días de temporada alta superan el límite superior de 37.718 €.

Tras la eliminación quedan **292 días** para el modelado.

7.6. Preprocesado de variables categóricas

La variable `DiaSemana` es categórica (Lunes, Martes, ..., Domingo). Se aplica **OneHotEncoding** para transformarla en columnas binarias, eliminando cualquier relación ordinal artificial. Se generan 6 columnas binarias (el dataset no contiene transacciones en sábado, por lo que este día no aparece).

7.7. Normalización

Se aplica **StandardScaler** sobre `GananciasTotales` y `Lag_1_Ventas` para que ambas variables tengan media 0 y desviación estándar 1. Las variables `DiaMes` y las columnas binarias de día de la semana no se normalizan. El dataset final queda así:

Cuadro 23: Primeras 5 filas del dataset final tras OneHotEncoding y estandarización (columnas de días de la semana en formato binario).

Fecha	GanTotales	DiaMes	Lag_1	Dom	Jue	Lun	Mar	Mié
2010-12-02	1,112	2	1,526	0	1	0	0	0
2010-12-03	1,002	3	0,942	0	0	0	0	0
2010-12-05	0,721	5	0,844	1	0	0	0	0
2010-12-06	2,497	6	0,593	0	0	1	0	0
2010-12-07	1,933	7	2,180	0	0	0	1	0

7.8. División en conjuntos de entrenamiento, validación y test

Se realiza una **división temporal**: primero se separa el período de test (últimas semanas del dataset), y luego se divide el resto en train (80 %) y validación (20 %), respetando el orden cronológico para no introducir fuga de datos del futuro al pasado.

Train: 220 filas (2010-12-02 -> 2011-09-02)
 Validation: 55 filas (2011-09-04 -> 2011-11-07)
 Test: 17 filas (2011-11-09 -> 2011-12-09)
 Total: 292 filas

Archivos exportados:

```
output/regresion/data_train_regresion.csv
output/regresion/data_validation_regresion.csv
output/regresion/data_test_regresion.csv
```

El conjunto de test cubre del 9 de noviembre al 9 de diciembre de 2011 (17 días), periodo de alta actividad que pone a prueba la capacidad de generalización del modelo fuera de su rango de entrenamiento.

8. Modelo Random Forest

El objetivo de este modelo es predecir las **ganancias totales diarias** a partir de las variables preprocesadas en la fase de tratamiento, utilizando el algoritmo **Random Forest Regressor**. Random Forest construye un conjunto de árboles de decisión entrenados sobre subconjuntos aleatorios de los datos y promedia sus predicciones, lo que reduce la varianza y mejora la generalización respecto a un único árbol.

Las librerías utilizadas son **Pandas** y **NumPy** para la manipulación de datos, **Matplotlib** para la visualización y **Scikit-learn** para el modelado y la evaluación de métricas (MAE, R^2 , RMSE).

8.1. Carga del dataset

Se cargan los tres conjuntos generados en la fase de tratamiento:

```
Train:      (220, 9)
```

```
Validation: (55, 9)
```

```
Test:       (17, 9)
```

```
Columnas: ['GananciasTotales', 'DiaMes', 'Lag_1_Ventas',  
           'DiaSemana_Domingo', 'DiaSemana_Jueves', 'DiaSemana_Lunes',  
           'DiaSemana_Martes', 'DiaSemana_Miercoles', 'DiaSemana_Viernes']
```

```
X_train shape: (220, 8)
```

```
y_train shape: (220,)
```

La variable objetivo es **GananciasTotales** (estandarizada). Las 8 características de entrada incluyen **Lag_1_Ventas**, **DiaMes** y las 6 columnas binarias de día de la semana.

8.2. Entrenamiento del modelo

Se entrena el modelo con los siguientes hiperparámetros, seleccionados para equilibrar capacidad expresiva y regularización:

- **n_estimators=100**: 100 árboles en el bosque.
- **max_depth=10**: profundidad máxima de cada árbol para limitar el sobreajuste.
- **min_samples_split=5**: mínimo de muestras para dividir un nodo interno.
- **max_features='log2'**: número de características consideradas en cada división.
- **random_state=42**: semilla fija para reproducibilidad.

8.3. Evaluación del modelo

Se evalúan las predicciones sobre los tres conjuntos con MAE, R^2 y RMSE. Todas las métricas operan sobre los valores estandarizados:

Cuadro 24: Métricas de evaluación del modelo Random Forest en los tres conjuntos (valores sobre datos normalizados).

Conjunto	MAE	R^2	RMSE
Entrenamiento	0,2724	0,8053	0,3471
Validación	0,8160	-0,4091	1,0396
Test	1,0414	-1,1823	1,1924

Modelo Random Forest entrenado correctamente

ENTRENAMIENTO:

MAE: 0.2724

R^2 : 0.8053

RMSE: 0.3471

VALIDACIÓN:

MAE: 0.8160

R^2 : -0.4091

RMSE: 1.0396

TEST:

MAE: 1.0414

R^2 : -1.1823

RMSE: 1.1924

El modelo obtiene un $R^2=0,81$ en entrenamiento, lo que indica una buena capacidad de ajuste sobre los datos conocidos. Sin embargo, los valores negativos de R^2 en validación $(-0,41)$ y test $(-1,18)$ revelan un claro **sobreajuste**: el modelo no logra generalizar a períodos no vistos durante el entrenamiento. El conjunto de test, que cubre las últimas 17 jornadas del año (noviembre-diciembre 2011), presenta una dinámica de ventas muy diferente al período de entrenamiento, lo que dificulta la predicción.

8.4. Importancia de variables

Random Forest calcula la importancia de cada variable según cuánto reduce la impureza en los nodos de los árboles durante el entrenamiento:

Cuadro 25: Importancia de las variables en el modelo Random Forest.

Variable	Importancia
Lag_1_Ventas	0,4142
DiaSemana_Domingo	0,2699
DiaMes	0,1904
DiaSemana_Jueves	0,0440
DiaSemana_Martes	0,0235
DiaSemana_Lunes	0,0227
DiaSemana_Viernes	0,0186
DiaSemana_Miercoles	0,0169

Importancia de features (Random Forest):

feature	importance
Lag_1_Ventas	0.4142
DiaSemana_Domingo	0.2699
DiaMes	0.1904
DiaSemana_Jueves	0.0440
DiaSemana_Martes	0.0235
DiaSemana_Lunes	0.0227
DiaSemana_Viernes	0.0186
DiaSemana_Miercoles	0.0169

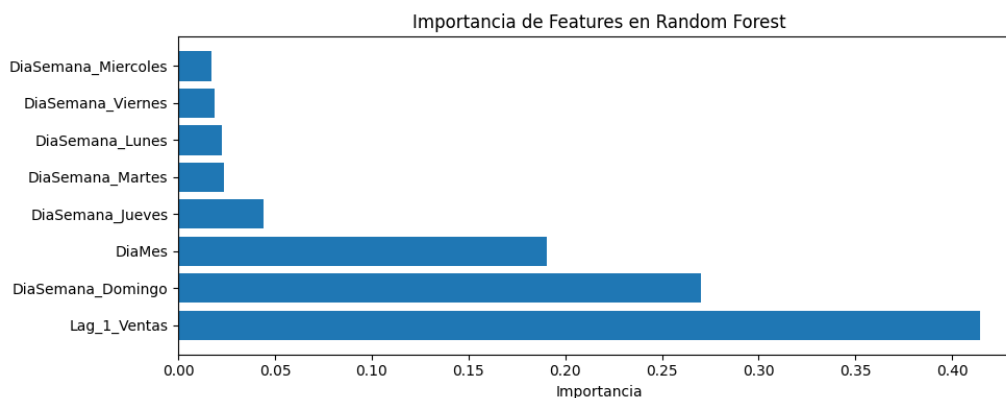


Figura 15: Importancia de las variables en el modelo Random Forest.

`Lag_1_Ventas` es la variable más influyente (0,41), seguida por `DiaSemana_Domingo` (0,27) y `DiaMes` (0,19). El peso del lag diario sugiere que la tendencia reciente de ventas es el predictor más poderoso. El protagonismo del domingo es notable, posiblemente reflejando los picos de facturación al inicio de la semana de trabajo.

8.5. Visualización de predicciones

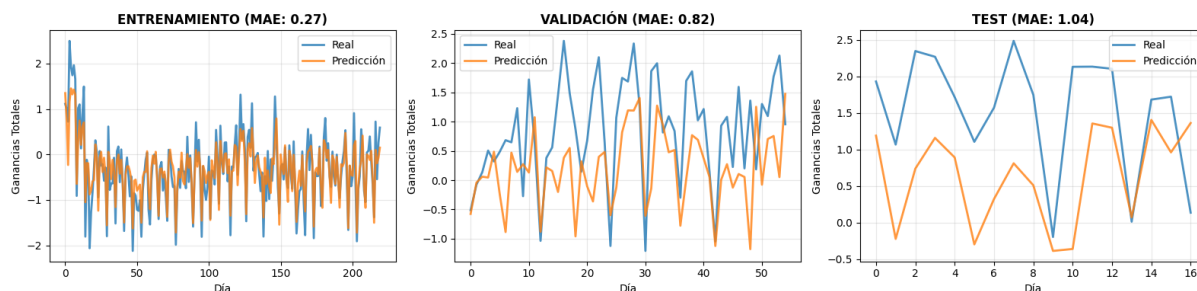


Figura 16: Predicciones vs valores reales del modelo Random Forest en los conjuntos de entrenamiento, validación y test (valores normalizados).

En el conjunto de entrenamiento el modelo sigue razonablemente bien la curva real. En validación y test las predicciones se alejan de los valores reales, confirmando el sobreajuste observado en las métricas. La capacidad limitada del modelo para capturar la tendencia creciente del período navideño es la principal fuente de error en el conjunto de test.

9. Modelo XGBoost

El objetivo de este modelo es predecir las **ganancias totales diarias** utilizando **XGBoost** (*Extreme Gradient Boosting*), un algoritmo de *gradient boosting* que combina múltiples árboles de decisión débiles de forma secuencial, donde cada árbol corrige los errores del anterior. XGBoost incorpora regularización L1 y L2 de forma nativa, lo que lo hace especialmente competitivo en datasets tabulares de tamaño reducido.

Las librerías utilizadas son **Pandas** y **NumPy** para la manipulación de datos, **Matplotlib** para la visualización, **XGBoost** para el algoritmo de regresión y **Scikit-learn** para las métricas de evaluación (MAE, R^2 , RMSE).

9.1. Carga del dataset

Se cargan los mismos tres conjuntos utilizados en el modelo Random Forest:

Train: (220, 9)

Validation: (55, 9)

Test: (17, 9)

Columnas: ['GananciasTotales', 'DiaMes', 'Lag_1_Ventas',
'DiaSemana_Domingo', 'DiaSemana_Jueves', 'DiaSemana_Lunes',
'DiaSemana_Martes', 'DiaSemana_Miercoles', 'DiaSemana_Viernes']

X_train shape: (220, 8)

y_train shape: (220,)

9.2. Entrenamiento del modelo

Se entrena el modelo con los siguientes hiperparámetros:

- `n_estimators=100`: número de rondas de boosting.
- `max_depth=5`: profundidad máxima de los árboles, menor que en Random Forest para mayor regularización.
- `learning_rate=0.05`: tasa de aprendizaje baja para un ajuste más gradual y estable.
- `subsample=0.8`: fracción de filas usadas en cada árbol para reducir varianza.
- `colsample_bytree=0.8`: fracción de columnas usadas en cada árbol.
- `random_state=42`: semilla fija para reproducibilidad.

9.3. Evaluación del modelo

Se evalúan las predicciones sobre los tres conjuntos con MAE, R^2 y RMSE:

Cuadro 26: Métricas de evaluación del modelo XGBoost en los tres conjuntos (valores sobre datos normalizados).

Conjunto	MAE	R^2	RMSE
Entrenamiento	0,2265	0,8675	0,2864
Validación	0,8690	-0,6606	1,1286
Test	1,2024	-1,8831	1,3705

Modelo XGBoost entrenado correctamente

ENTRENAMIENTO:

MAE: 0.2265
 R^2 : 0.8675
 RMSE: 0.2864

VALIDACIÓN:

MAE: 0.8690
 R^2 : -0.6606
 RMSE: 1.1286

TEST:

MAE: 1.2024
 R^2 : -1.8831
 RMSE: 1.3705

El modelo obtiene un $R^2=0,87$ en entrenamiento, superior al de Random Forest, lo que indica mayor capacidad de ajuste sobre los datos de entrenamiento. Sin embargo, el sobreajuste es también más pronunciado: en validación $R^2=-0,66$ y en test $R^2=-1,88$, valores peores que los obtenidos por Random Forest. El mecanismo de boosting, que optimiza iterativamente sobre los residuos, tiende a memorizar más el conjunto de entrenamiento cuando el número de muestras es pequeño (220 días), lo que perjudica la generalización.

9.4. Importancia de variables

XGBoost calcula la importancia de cada variable según cuánto contribuye a reducir la función de pérdida durante el entrenamiento:

Cuadro 27: Importancia de las variables en el modelo XGBoost.

Variable	Importancia
DiaSemana_Domingo	0,5505
DiaSemana_Jueves	0,0971
DiaSemana_Martes	0,0778
Lag_1_Ventas	0,0744
DiaSemana_Lunes	0,0708
DiaSemana_Viernes	0,0504
DiaMes	0,0410
DiaSemana_Miercoles	0,0381

Importancia de features (XGBoost):

feature	importance
DiaSemana_Domingo	0.5505
DiaSemana_Jueves	0.0971
DiaSemana_Martes	0.0778
Lag_1_Ventas	0.0744
DiaSemana_Lunes	0.0708
DiaSemana_Viernes	0.0504
DiaMes	0.0410
DiaSemana_Miercoles	0.0381

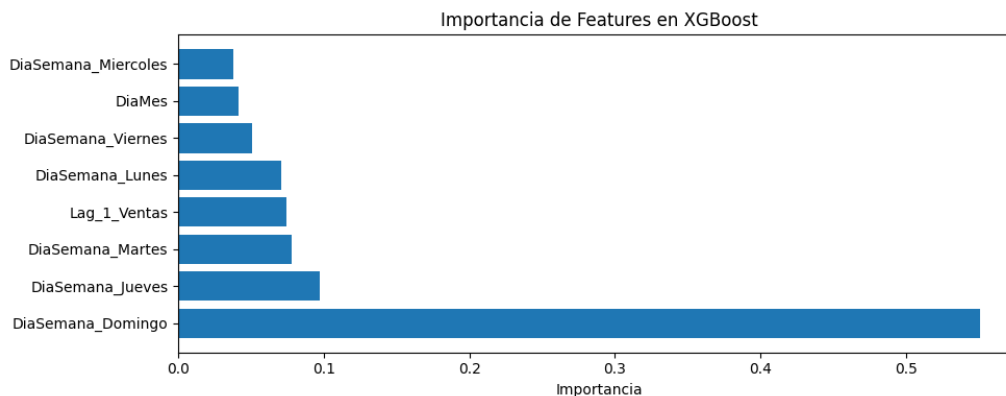


Figura 17: Importancia de las variables en el modelo XGBoost.

A diferencia de Random Forest, XGBoost otorga el mayor peso a `DiaSemana_Domingo` (0,55), relegando `Lag_1_Ventas` al cuarto lugar (0,07). Esta diferencia en las importancias entre ambos modelos refleja que cada algoritmo extrae señales distintas del mismo conjunto de características: Random Forest prioriza la autocorrelación temporal (lag), mientras que XGBoost se apoya más en el patrón semanal.

9.5. Visualización de predicciones

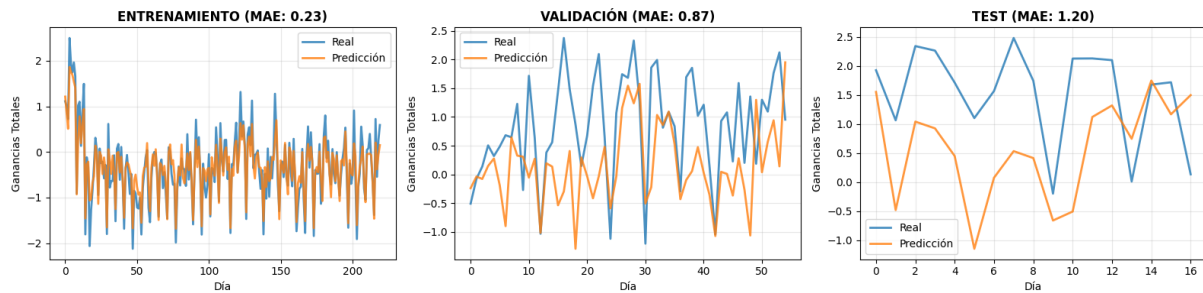


Figura 18: Predicciones vs valores reales del modelo XGBoost en los conjuntos de entrenamiento, validación y test (valores normalizados).

En entrenamiento el modelo sigue la curva real con precisión. En validación y test la capacidad predictiva se deteriora de forma más acusada que en Random Forest, especialmente en el conjunto de test, donde los 17 días del período navideño presentan una dinámica de ventas muy distinta a la observada durante el entrenamiento.

10. Conclusiones

Este trabajo ha abordado dos problemas distintos sobre el mismo conjunto de datos de e-commerce: la **segmentación de clientes** mediante algoritmos de clustering y la **predicción de ventas diarias** mediante modelos de regresión. A continuación se recogen las conclusiones principales de cada fase.

Exploración y limpieza

El dataset original presentaba una calidad moderada: duplicados, registros con **Quantity** o **UnitPrice** negativos, **CustomerID** nulos y outliers extremos. Tras la limpieza se conservaron 355.996 transacciones válidas para clustering y 476.888 para regresión, lo que demuestra que incluso con una depuración rigurosa el volumen de datos resultante es suficientemente representativo.

Clustering

K-Means con $K = 4$ ofrece la mejor relación entre interpretabilidad y calidad de la segmentación (Silhouette Score = 0,46, Davies-Bouldin = 1,03). Los cuatro clusters identifican perfiles nítidos: clientes ocasionales, activos, diversificados y muy activos, con diferencias claras en transacciones, gasto y frecuencia de compra. La proyección PCA captura el 93 % de la varianza, lo que confirma que las cuatro variables derivadas concentran la información relevante del comportamiento de los clientes.

DBSCAN con $\text{eps}=0,75$ y $\text{min_samples}=8$ genera 8 clusters con solo un 1,3 % de ruido, produciendo una segmentación más granular (6 niveles de actividad). Sus métricas son inferiores a las de K-Means (Silhouette = 0,25, Davies-Bouldin = 2,57), lo que es esperable dado que DBSCAN opera por densidad y produce fronteras más difusas. Su valor principal es la detección automática de outliers y la capacidad de encontrar clusters de forma irregular, sin necesidad de especificar K de antemano.

Ambos modelos coinciden en señalar que el grueso de los clientes (más del 50 %) pertenece al segmento de menor actividad, mientras que los clientes de alto valor son una minoría que concentra gran parte del negocio.

Regresión

Los modelos de regresión presentan el reto más significativo del proyecto. Tanto **Random Forest** ($R^2_{\text{train}}=0,81$, $R^2_{\text{test}}=-1,18$) como **XGBoost** ($R^2_{\text{train}}=0,87$, $R^2_{\text{test}}=-1,88$) muestran un claro sobreajuste al dato de entrenamiento, con pérdida de capacidad predictiva en validación y test.

Las causas principales son estructurales: el dataset temporal solo dispone de 292 días tras la limpieza, de los cuales únicamente 17 conforman el conjunto de test, correspondientes al período navideño de 2011, cuya dinámica de ventas difiere notablemente del resto del año. Con tan poco volumen temporal y una variable objetivo de alta variabilidad diaria, ambos modelos no disponen de suficiente señal para generalizar correctamente.

En cuanto a la importancia de variables, ambos algoritmos coinciden en que el patrón semanal y la tendencia reciente (lag del día anterior) son los predictores más relevantes, aunque difieren en su peso relativo: Random Forest prioriza el lag diario, mientras que XGBoost prioriza el día de la semana.

Valoración global

El clustering ha demostrado ser la parte más sólida del proyecto, produciendo segmentaciones estables e interpretables desde el punto de vista del negocio. La regresión, por su parte, pone de manifiesto las limitaciones inherentes a series temporales cortas con alta estacionalidad: para obtener modelos robustos sería necesario contar con varios años de histórico, incorporar variables externas (festivos, campañas) y explorar arquitecturas específicas para series temporales como LSTM o Prophet.

Referencias

Dataset

1. Kaggle. *E-commerce Data*. Disponible en: <https://www.kaggle.com/datasets/carrie1/ecommerce-data/>

Librerías utilizadas

1. NumPy. <https://numpy.org/>
2. Pandas. <https://pandas.pydata.org/>
3. Matplotlib. <https://matplotlib.org/>
4. Scikit-learn. <https://scikit-learn.org/>
5. XGBoost. <https://xgboost.readthedocs.io/>
6. K-Means. <https://scikit-learn.org/stable/modules/clustering.html#k-means>
7. DBSCAN. <https://scikit-learn.org/stable/modules/clustering.html#dbscan>
8. Random Forest.
<https://scikit-learn.org/stable/modules/ensemble.html#random-forests>

10.1. Sobre el documento

En la elaboración de este documento se ha utilizado inteligencia artificial como herramienta de apoyo para mejorar la legibilidad, claridad y estructuración del contenido, así como para la generación automatizada de elementos complejos como tablas para agilizar el proceso.

No obstante, la redacción, los análisis de datos y el proceso de modelado predictivo fueron realizados manualmente en Jupyter Notebook, asegurando la validez metodológica y precisión de los resultados obtenidos.

10.2. Repositorio y código fuente

Código fuente del proyecto. Disponible en: https://github.com/saulbasc/Regresion_y_Clustering_E-Commerce